

Final Paradigmas Lab OOP

5 de Agosto 2026

Instrucciones Generales

- Para codear el examen solo podrán utilizar algún editor de texto plano o IDE (sin autocompletado con IA). No utilizar teléfonos durante el examen.
- Pueden buscar información en Google pero no pueden utilizar asistentes de IA de ninguna clase. Es obligatorio deshabilitar los resúmenes de IA que ofrece google introduciendo el comando “-IA” en todas las consultas a Google. Alternativamente pueden utilizar un buscador clásico como ecosia.
- Pueden utilizar sus propias laptops o alguna máquina del laboratorio que tenga instalado sbt.
- Pueden tener el código fuente del laboratorio para consulta pero NO deben entregarlo.-Solo deben entregar el archivo Main.scala provisto en el esqueleto
- El código que implementan ustedes debe cumplir que **todas las componentes sean declarativas, i.e., funcionales puras.**

El esqueleto contiene dos proyectos: el proyecto principal del examen y un servidor mock de Reddit ([reddit-mock/](#)). **Deben tener ambos corriendo al mismo tiempo**, cada uno en una terminal separada.

Link para descargar el skeleton: [LINK](#)

Para descomprimir: `tar -xzf skeleton.tar.gz`

Paso 1 — Levantar el servidor mock de Reddit

Para evitar que Reddit bloquee las peticiones durante el examen, el esqueleto apunta a un servidor local que simula la API de Reddit.

En una terminal, desde la carpeta [reddit-mock/](#):

```
None  
sbt run
```

Deberían ver:

None

Fake Reddit API running on <http://localhost:8123>
Press Ctrl+C (or stop the process) to shut down.

El servidor queda corriendo en el puerto 8123 y sirve los posts de los subreddits [r/scala](#), [r/programming](#) y [r/learnpython](#) desde archivos JSON locales. **No cerrarlo durante el examen.**

Paso 1 — Compilar el proyecto

Para ejecutar el esqueleto solo tienen que descomprimirlo y correr [sbt run](#). Verán algo como el siguiente output.

```
(sin entidades detectadas)
Post: "Clock Synchronization Is a Nightmare"
Entidades detectadas:
(sin entidades detectadas)
Post: "While GitHub Actions remains a key part of this vision, we are allocating resources towards other areas ..."
Entidades detectadas:
[Organization] GitHub
Post: "A Dab of DuckDB"
Entidades detectadas:
(sin entidades detectadas)
Post: "A Dab of DuckDB"
Entidades detectadas:
(sin entidades detectadas)
Post: "Modern LZ Compression Part 2: FSE and Arithmetic Coding"
Entidades detectadas:
(sin entidades detectadas)
Post: "How the Lobsters front page works - nilenso blog"
Entidades detectadas:
(sin entidades detectadas)
Post: "Engineering Health Essentials"
Entidades detectadas:
(sin entidades detectadas)
Post: "Hunting a Windows ARM crash through Rust, C, and a Build-System configurations"
Entidades detectadas:
[ProgrammingLanguage] Rust
Post: "How to become a better Unity C# Programmer"
Entidades detectadas:
(sin entidades detectadas)
Post: "Bitwarden CLI Compromised in Ongoing Checkmarx Supply Chain"
Entidades detectadas:
[ProgrammingLanguage] Go

=== Estadísticas de entidades ===
ProgrammingLanguage: 48
Organization: 4
Place: 1
[success] Total time: 3 s, completed Aug 5, 2026, 11:04:18 AM
➔ skeleton
```

Ejercicio 1

Cuando el matching se hace por simple coincidencia de substring aparecen falsos positivos: el lenguaje "Go" aparece como substring dentro de palabras como "Google" o "Django", y "Java" aparece como substring de "JavaScript". El ejercicio consiste en que los lenguajes de programación exijan una coincidencia de palabra completa para considerarse detectados, sin modificar el comportamiento del resto de los tipos.

- a) Agregar en **NamedEntity** el método concreto:

None

```
def appearsIn(content: String): Boolean
```

que implemente el comportamiento por defecto que hoy tenemos en **Analyzer.detectEntities**, es decir, una coincidencia de substring.

- b) Sobreescibir **appearsIn** en **ProgrammingLanguage** para que exija coincidencia de palabra completa (case-insensitive) en lugar de substring.

Por ejemplo, para el lenguaje "Go": el texto "Check out this repo on Google" NO debe matchear, pero "Go is a great language" SÍ. Para "Java": "I love JavaScript" NO debe matchear, pero "I'm learning Java" SÍ. (Sugerencia: usar una expresión regular).

- c) Modificar **Analyzer.detectEntities** para que use **appearsIn** en lugar de la comparación que tenía hardcodeada.

Ejercicio 2

Actualmente el reporte de estadísticas finales usa el nombre técnico de la clase (Person, Organization, etc.) tal cual. El ejercicio consiste en que cada tipo de entidad tenga su propia forma de describirse en el reporte, en español y con la pluralización correcta.

- a) Agregar en **NamedEntity** el método concreto:

None

```
def pluralLabel(count: Int): String
```

que como comportamiento por defecto: usa el nombre técnico de la clase tal cual, sin traducir. Es decir, devuelve el count y el nombre técnico.

Luego, sobreescibir **pluralLabel** en cada uno de los siguientes tipos, devolviendo la redacción en español con la pluralización correcta (singular si count es 1, plural en caso contrario):

Tipo	count = 1	count > 1
Person	1 persona	N personas
Organization	1 organización	N organizaciones
University	1 universidad	N universidades

Place	1 lugar	N lugares
Technology	1 tecnología	N tecnologías
ProgrammingLanguage	1 lenguaje de programación	N lenguajes de programación

b) Agregar en *Analyzer* la función:

None

```
def summarizeByLabel(entities: List[NamedEntity]): List[String]
```

que agrupe las entidades detectadas por tipo y devuelva, por cada grupo, la cadena generada por *pluralLabel* de ese tipo (aplicado a la cantidad de entidades del grupo), ordenadas de mayor a menor cantidad.

Para construir cada línea alcanza con tomar cualquier entidad del grupo como representante e invocar su *pluralLabel(count)*, ya que el resultado depende del tipo, no de la instancia puntual. Hint: pueden utilizar *group.head* para obtener el primer elemento.

c) Integrar la llamada a *Analyzer.summarizeByLabel* en el main del programa: después de imprimir las estadísticas existentes (*formatEntityStats*), agregar la impresión del resumen generado por *summarizeByLabel* sobre las mismas entidades detectadas (*allDetected*).

Ejemplo de salida esperada (con datos ilustrativos):

None

```
=== Resumen por tipo ===
```

```
45 lenguajes de programación
```

```
4 organizaciones
```

```
1 lugar
```

Ejercicio 3

Hasta acá, cada entidad se detecta o no dentro de un post individual. En este ejercicio vamos a identificar qué entidades son "tendencia": las que aparecen mencionadas en varios posts distintos, no en uno solo.

a) Agregar en Analyzer la función:

None

```
def trendingEntities(posts: List[String], dictionary: List[NamedEntity], minMentions: Int):  
  List[NamedEntity]
```

que, dada una lista de títulos de posts (texto plano) y el diccionario completo de entidades, devuelva las entidades que aparecen (según *appearsIn*, el mismo método del Ejercicio 1) en al menos **minMentions** posts distintos. **minMentions** se pasa como parámetro, no debe estar hardcodeado.

Por ejemplo: si *minMentions* = 3 y "Python" aparece (según *appearsIn*) en 4 de los posts analizados, "Python" debe estar en el resultado. Si "Rust" aparece en 1 solo post, no debe estar.

Aclaración: "aparece en un post" significa que *entity.appearsIn(postTitle)* es true para ese post — no importa cuántas veces se repita el nombre dentro de un mismo título, cuenta como una sola mención de ese post.

b) Integrar la llamada a *Analyzer.trendingEntities* en el main del programa. Para eso hace falta la lista completa de títulos de los posts descargados (no solo las entidades detectadas); se puede armar a partir de las mismas suscripciones, por ejemplo:

Java

```
val allTitles: List[String] = subscriptions.flatMap { url =>  
  val json = FileIO.downloadFeed(url)  
  FileIO.extractPostTitles(json)  
}
```

Con esa lista, invocar *trendingEntities* con el valor de **minMentions** que prefieran (por ejemplo, 3) e imprimir las entidades resultantes.

Entrega

Para entregar el examen deben subir **TODOS los archivos modificados por ustedes** a través del formulario de entrega adjuntando un solo archivo comprimido que los incluya a todos:

Para generar el archivo comprimido pueden usar un comando **similar** al siguiente:

None

```
tar -czvf fulano.tar.xz src/main/scala/*.scala
```

donde * son solo los archivos modificados. **Recomendado verificar que estén todos los archivos que se modificaron luego de generar el comprimido.**

Deberán recibir un mail de confirmación, no se retiren del aula hasta recibirlo, ya que, este representa su único comprobante de entrega de examen.